

# 先從 Unity 熟悉的畫面開始

## Unity 常見 Scene

### Player GameObject

PlayerInput、角色控制 Script、Camera  
常見做法：先放進 Scene

### EnemySpawner GameObject

Spawner Script  
負責在執行時產生 Enemy

## Unreal : Variant\_Combat

### 玩家：由 GameMode 提供類別

Lvl\_Combat → BP\_CombatGameMode  
→ PlayerController 控制 Character

### 敵人：由關卡中的 Spawner 生成

Lvl\_Combat → CombatEnemySpawner  
→ CombatEnemy

先看責任：誰放在關卡裡？誰在執行時才建立？

# Lvl\_Combat 裡實際放了什麼



關卡先放：GameMode 設定、PlayerStart、EnemySpawner；玩家與 Enemy 之後才建立。

# 玩家不是預先放好的，而是依 GameMode 建立

設定：GameMode 指定要用的類別

**PlayerController Class**

BP\_CombatPlayerController

**Default Pawn Class**

BP\_CombatCharacter

執行：進入 PIE 後才建立物件

**PlayerStart**

選出生點

**建立兩個物件**

Controller +  
Character

**開始控制**

Controller Possess  
Character

**PlayerController 與 Character 是不同物件**

GameMode 提供兩個類別；執行時建立後，再由  
PlayerController 控制 Character。

Possess = Controller 開始控制 Pawn；Character 因為是 Pawn，所以能被控制。

# EnemySpawner 放在關卡裡，Enemy 執行時才出現



## 這條生怪流程不經過 GameMode

EnemySpawner 是放在 Lvl\_Combat 裡的 AActor；它在事件觸發後生成 CombatEnemy。

# Unity 與 Unreal 都有 Component

## Unity

**Camera GameObject**

場景中的物件容器



**Camera Component**

真正提供相機能力

## Unreal

**ACameraActor**

Unreal 預先組好的 Actor



**UCameraComponent**

Actor 內仍然靠 Component 提供相機能力

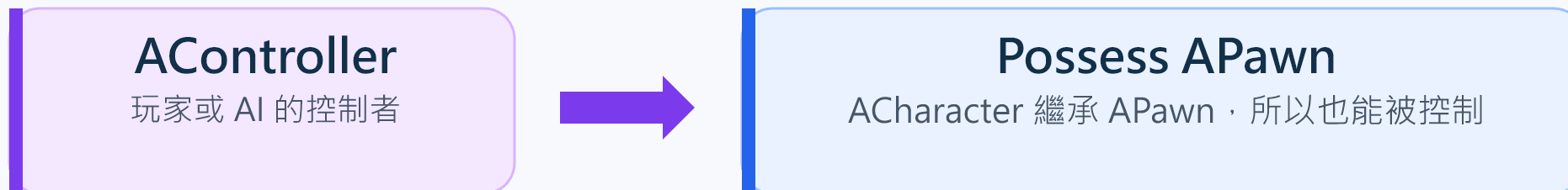
兩套引擎都有 Component ; Unreal 常把常用 Component 預先組成現成 Actor 。

# UE 常把 Component 組成現成 Actor

| 用途     | Unity 常見組合                             | Unreal 現成 Actor                            |
|--------|----------------------------------------|--------------------------------------------|
| Camera | GameObject<br>+ Camera Component       | ACameraActor<br>+ UCameraComponent         |
| Light  | GameObject<br>+ Light Component        | APointLight<br>+ UPointLightComponent      |
| 靜態模型   | GameObject<br>+ Mesh Filter / Renderer | AStaticMeshActor<br>+ UStaticMeshComponent |

Actor 子類別是可直接使用的常見組合；Component 仍然可以加入其他 Actor。

# Actor、Pawn、Character 怎麼分



CombatEnemySpawner = AActor

CombatCharacter / CombatEnemy = ACharacter

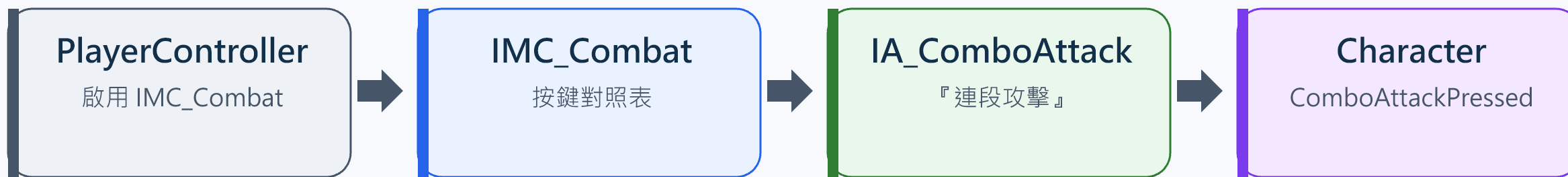
一般 AActor 不能被 Possess，但仍然可以被程式或 Component 操作。

## 先看結果：左鍵觸發連段攻擊



左鍵不會直接呼叫攻擊程式；它先被轉成 IA\_ComboAttack。

## 先分工作：誰啟用、誰配鍵、誰做事



遊戲開始時啟用整張表

LeftMouseButton

Gamepad Right Shoulder

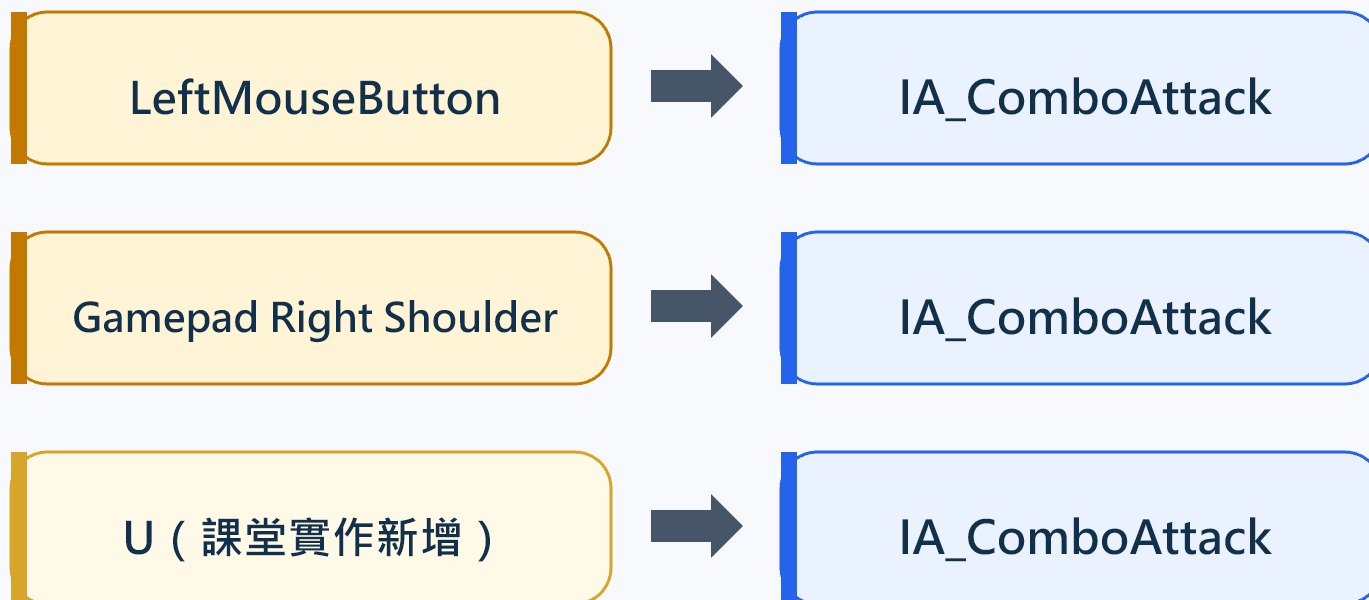
兩顆按鍵都在 IMC 裡對到同一個 IA

看起來像設定很多次，其實每個位置只回答一個問題。

PC：用哪張表 | IMC：哪顆鍵 | IA：什麼動作 | Character：收到後做什麼

# IMC 配按鍵，IA 不記得是哪顆按鍵

## IMC\_Combat : Mappings



## IA\_ComboAttack

Value Type : Boolean

只表示：  
玩家要連段攻擊

不知道玩家用左鍵、手把，還是  
U 鍵

IA 不是按鍵，也不是攻擊程式；它是兩邊共同認得的動作名稱。

# Character 綁定 IA，收到後才做攻擊

初始化時只綁定一次

SetupPlayerInputComponent : IA\_ComboAttack + Started → ComboAttackPressed



**DoComboAttackStart**

先判斷角色目前是否正在攻擊



正在攻擊

**記錄這次輸入時間**

等待接續下一段連段



尚未攻擊

**ComboAttack**

Montage\_Play → AM\_ComboAttack

Character 綁的是 IA\_ComboAttack，不是 LeftMouseButton。

## 為什麼要拆開？因為改一處就夠了



三種事情會因不同原因改變，所以拆成三個責任；IA 是中間共同的動作名稱。

## 示範 2 總結：四個責任

1

**PlayerController**

啟用目前要使用的 IMC。

2

**IMC**

設定哪些實體按鍵對應哪些 IA。

3

**IA**

代表玩家想做的動作，不綁死某一顆按鍵。

4

**Character**

綁定 IA，收到後執行角色行為。

**按鍵 → IMC → IA → Character**